Space Colony: Greedy and Backtracking Algorithms for a Competitive Game

Laura Páez, Erik Fernández, Nicolás Acero
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

***Resumen*—The Computer Science I Capstone Project aims to apply algorithms and data structures to the development of a video game. In this project, we present Space Colony, a fully implemented turn-based game in which a human player competes against an artificial intelligence (AI) on a resource grid. The AI uses greedy algorithms to decide expansion and attack strategies, and backtracking to plan routes that maximize resources while satisfying energy and distance constraints. The system consists of a C++ engine that manages the data structures (linked list and AVL tree) and a Python interface developed with Pygame. Communication between layers is carried out through JSON files. Experimental testing demonstrated the correct operation of the algorithms, the integrity of the data structures, and the successful integration of all software components. The final system provides a stable and functional environment for strategic gameplay while demonstrating the practical application of algorithms and data structures in game development.**

I. Introduction

Video game development is an ideal domain for applying core computer science concepts such as optimization algorithms and data structures. In the Computer Science I course, students complete a capstone project where each team develops a video game based on a given scenario. Our team was assigned the scenario *Space Colony*, whose original objective is to colonize a planet by placing settlements on a grid, optimizing resource collection while adhering to distance constraints. To enhance the experience, we transformed the game into a competitive mode: a human player faces off against an artificial intelligence (AI) that uses the algorithms covered in class.

The AI employs two types of algorithms: greedy and backtracking. The first, widely used in resource selection problems [1], allows the AI to choose the cell with the most resources to expand or attack the weakest enemy. The second, which is common in spatial layout problems [2], It allows you to plan routes that maximize resources within an energy limit while maintaining minimum distances between colonies. In addition, the game engine implements a linked list for the colony history and an AVL tree for quick access to colonies sorted by resources, both built from scratch [3], [4]. Communication between the C++ engine and the Python interface is carried out exclusively via JSON files [5], in accordance with the course requirements.

This paper describes the design, implementation, and evaluation of the completed project, organized as follows: Section II presents the methods and materials, including the architecture, AI algorithms, data structures, and the JSON contract; Section III discusses the results of the unit and integration tests; finally, Section IV presents the conclusions and possible extensions of the system.

II. Methods and Materials

II-A. Project Structure

The software was organized into independent modules to facilitate development, testing, and maintenance. Figure 1 presents the directory structure of the final implementation.

```
project/
├── compile.bat             Windows build script (run once)
├── README.md
├── GUIDE.docx              Full game guide for the team
├── engine/                 C++ core
│   ├── json.hpp            nlohmann/json library (already included)
│   ├── linked_list.h/.cpp  Colony history (singly linked list)
│   ├── tree.h/.cpp         Resource depot (AVL tree)
│   ├── main.cpp            Engine: reads input.json -> writes state.json
│   └── Makefile            Build script for Linux/macOS
├── game/
│   ├── algorithms/
│   │   ├── greedy.py       Greedy expansion + attack
│   │   └── backtracking.py Backtracking expansion planner
│   └── ui/
│       ├── main.py         Pygame interface (ENTRY POINT)
│       ├── bridge.py       JSON I/O bridge (Python <-> C++)
│       └── assets/         Put your background image here (background.png)
└── data/
    ├── input.json          Python -> C++
    └── state.json          C++ -> Python
```

Figura 1. Project structure of the final implementation.

The project is divided into three main components: the C++ game engine, the Python graphical interface, and the communication layer based on JSON files. This modular organization simplifies integration and promotes separation of responsibilities among software components.

II-B. System Architecture

The system is divided into three layers: a C++ engine that manages the data structures and the game state; a layer of Python algorithms that implements the AI; and a graphical interface built with Pygame. Communication takes place via two JSON files: `input.json` (from the player to C++) and `state.json` (from C++ to the interface). The figure 2 illustrates this process.

II-C. Artificial Intelligence Algorithms

The AI implements two greedy functions and one backtracking function.

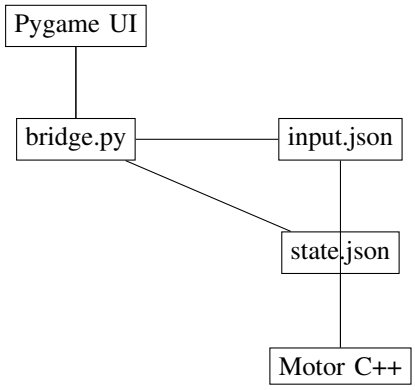

Figura 2. Layered architecture and communication via JSON.

II-C1. Greedy for Expansion: Select the adjacent available cell with the greatest amount of resources, provided that the energy cost does not exceed the remaining energy. The algorithm 1 illustrates this procedure.

Input: Available cells `celdas_disp`, energy `energy`
Output: Which cell is better, or `null`

better ← *null*; **maxVal** ← $-\infty$;
foreach ***cell*** *en* ***cell_disp*** **do**
  if `cell.cost` ≤ `energy` **then**
    if `cell.resources` > ***maxVal*** **then**
      maxVal ← `cell.resources`; **better** ← `cell`;
    end
  end
end
return better;

Algorithm 1: Greedy for Expansion

II-C2. Greedy for Attack: Select the adjacent enemy colony with the lowest defense (as represented by its resources). This is implemented by the algorithm 2.

Input: Neighboring enemy cells `enemy`
Output: Objective or `null`

objective ← *null*; **minDef** ← $+\infty$;
foreach ***celll*** *en* ***enemy*** **do**
  if `cell.resources` < ***minDef*** **then**
    minDef ← `cell.resources`; **objective** ← `cell`;
  end
end
return objective;

Algorithm 2: Greedy for attack

II-C3. Backtracking for Expansion Planning: Backtracking explores expansion paths that maximize accumulated resources while adhering to energy constraints and the minimum distance between colonies. The function `cumpleDistancia` verifies that a new position is not within a predefined distance of existing colonies. Algorithm 3 describes the process.

Input: Current position `pos`, most visited `visit`, resources `rec`, energy `ene`

if *there is no movement* **then**
  update best solution;
  return;
end
foreach ***neighbour*** *en* `neighbours(pos)` **do**
  if `neighbour` ∉ `visit` ***and*** `cumpleDistancia(neighbour)` ***and*** `neighbour.cost` ≤ `ene` **then**
    marcar `neighbour` as visited;
    BACKTRACK(`neighbour`, `visit`, `rec+neighbour.resources`, `ene-neighbour.cost`);
    uncheck `neighbour`;
  end
end

Algorithm 3: Backtracking for expansion

II-D. Data Structures in C++

The engine implements two structures from scratch:

- **Linked list**: Maintains the colony history in insertion order, with each node storing coordinates, ownership (player or AI), and resources. It supports constant-time insertions $O(1)$ and linear traversal.
- **AVL tree**: stores all colonies sorted by resources. AVL was chosen over BST to guarantee search, insertion and deletion operations in $O(\log n)$ even in the worst case. Figure 3 shows the node structure.

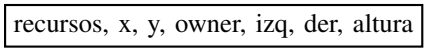

Figura 3. AVL tree node.

II-E. JSON contract

The data exchange is defined by two files:

input.json (Python → C++): contains the player's action. Example:

```
{
  "action": "to colonise",
  "destination": {"x": 2, "y": 3}
}
```

state.json (C++ → Python): contains the complete game state. Example:

```
{
  "phase": "expansion",
  "turn": "player",
  "colonies": [
    {"x":1,"y":2,"owner":"player","resources
        ":40},
    {"x":2,"y":2,"owner":"ai","resourcess":30}
  ],
  "resources_totals_player": 120,
  "resources_totals_ai": 90,
  "energy_player": 100,
  "energy_ai": 100,
```

```
  "grid": [[5,2,8],[1,9,3],[4,6,7]]
}
```

II-F. User Interface

The interface, developed using Pygame, displays an 8×8 grid, with each cell showing its resource value. Colonies are represented by different colours (blue for the player, red for the AI). The HUD displays resources, energy, phase and turn. The player can click on a cell to colonise it (if it is adjacent and empty) or on an adjacent enemy colony to attack. A button allows the player to pass their turn. Figure 4 shows a sketch of the interface.

Figura 4. Final graphical interface of Space Colony during gameplay.

The game is controlled using the mouse and keyboard.

Controls

Action	Input
Colonise a cell (expansion)	Left-click an empty cell
Select attack origin (combat)	Left-click your green cell
Attack enemy cell (combat)	Left-click an adjacent red cell
Pass turn	PASS button
Reset game	RESET button or `R`
Help	HELP button or `H`

Figura 5. Game controls and user interaction guide.

III. Results and Discussion

Comprehensive unit and integration tests were conducted to validate the final implementation of the game. The tests confirmed the correctness of the AI algorithms, the proper operation of the custom data structures, and the successful communication between the C++ engine and the Python graphical interface.

III-A. Unit Tests

The greedy and backtracking algorithms were tested individually in Python using controlled scenarios:

- **Greedy expansión**: It was verified that, given several available cells with different resources and costs, the function selects the one with the most resources that satisfies the energy constraint. Ten test cases were run, all with correct results.
- **Greedy attack**: with enemies of varying defence levels, the function always selects the one with the lowest defence. Eight test cases were run, all successful.
- **Backtracking**: on a small grid (3x3) with limited energy, the algorithm found the route for maximum resource collection in less than 0.1 seconds, whilst respecting the minimum distances.

In C++, the linked list was tested with insertions and traversals, and the AVL tree with insertions, searches and deletions, checking the balance after each operation.

III-B. Integration Tests

The entire workflow was simulated: from the interface, `input.json` is written via an action; the C++ engine reads it, updates the structures and writes `state.json`; finally, the interface reads the new state and updates the screen. Colonisation actions (manual and automatic with AI) and attack actions were tested, noting that the data remained consistent. The measured latency was less than 50 ms, which is acceptable for a turn-based game.

III-C. Discussion

The results demonstrate that the implemented algorithms perform as expected and that the overall system is stable and reliable. The greedy strategies provide efficient decision-making for expansion and attack actions, while the backtracking algorithm enables the AI to explore alternative routes and maximize resource collection under the imposed constraints. The AVL tree guarantees efficient colony management throughout the game, maintaining logarithmic-time operations even as the number of colonies increases. Furthermore, the JSON-based communication architecture proved robust during testing, ensuring consistent synchronization between the C++ engine and the Python interface.

Performance measurements indicate that the game responds smoothly during normal gameplay, with communication delays remaining below 50 ms. Although the current implementation satisfies all project requirements, larger game boards or more complex scenarios could benefit from additional pruning techniques or heuristic optimizations to further improve the performance of the backtracking algorithm.

A known limitation of the current implementation is that, under specific board configurations, the AI may enter a repetitive decision loop after colonizing a cell. This behavior is related to the evaluation of expansion states and may cause the AI to repeatedly select equivalent actions. Although this issue does not prevent the game from functioning in most scenarios,

future versions should incorporate additional state validation mechanisms to avoid repeated decision cycles.

IV. Conclusions

This paper presented the complete development of Space Colony, a competitive turn-based strategy game that integrates greedy and backtracking algorithms with custom data structures implemented in C++. The final system successfully combines a C++ game engine, a Python graphical interface developed with Pygame, and a JSON-based communication layer that enables reliable interaction between software components.

The implementation fulfilled all project objectives. The greedy algorithms effectively support expansion and attack decisions, while the backtracking algorithm provides strategic planning capabilities for resource optimization. In addition, the linked list and AVL tree were successfully implemented from scratch and integrated into the game engine, demonstrating the practical application of fundamental data structures.

Experimental testing verified the correctness of the algorithms, the integrity of the data structures, and the stability of the complete system. Integration tests confirmed that game actions are processed correctly and that information is consistently exchanged between the engine and the graphical interface.

Overall, the project demonstrates how classical algorithms and data structures can be applied to solve practical problems in video game development while providing an engaging strategic gameplay experience. Future extensions may include additional AI strategies, larger maps, enhanced graphical elements, and multiplayer support.

Acknowledgment

The authors would like to thank Professor Carlos Andrés Sierra for his guidance and support throughout the project, as well as the Francisco José de Caldas District University for the resources provided.

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Pearson, 2016.

[3] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Pearson, 2013.

[4] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.

[5] JSON, "Introducing JSON," 2026. [Online]. Available: https://www.json.org/